

ConversAI — Documentazione

Sviluppatore: [Luca Pes] [168358]

ConversAI è un'applicazione web per gestire conversazioni con un assistente virtuale, organizzate in progetti; ogni conversazione ha un proprio argomento (topic). L'applicazione è stata sviluppata con Node.js ed Express per il backend, SQLite come database, e HTML/CSS/JavaScript per il frontend. Si è scelto di implementare Bootstrap 5 per lo stile e l'interattività andando a fornire una migliore esperienza utente e un layout responsive senza dover scrivere tutto il CSS da zero. L'applicazione è progettata per funzionare anche offline, copiando le librerie necessarie in locale invece di fare affidamento su CDN esterni come da specifiche.

1. Panoramica delle funzionalità

- **Progetti:** creazione, rinomina ed eliminazione di progetti, ciascuno con un nome e una descrizione (facoltativa). Eliminare un progetto elimina automaticamente anche tutte le sue conversazioni e i relativi messaggi.
- **Conversazioni:** ogni progetto può contenere più conversazioni, ciascuna con un argomento (topic). Anche le conversazioni si possono rinominare ed eliminare; eliminarle cancella tutti i loro messaggi.
- **Messaggi e risposta simulata:** all'interno di una conversazione si possono scrivere messaggi. Per ognuno, il server genera automaticamente una risposta "dell'assistente" tramite delle regole prestabilite: riconosce saluti, ringraziamenti, richieste di aiuto, domande (punto di domanda finale), oppure fa un eco del messaggio ricevuto.
- **Interfaccia:** tema scuro con colore primario personalizzato (bordeaux), sidebar con l'elenco progetti/conversazioni (accordion su desktop, menu a scomparsa — offcanvas — su mobile), area messaggi con distinzione tra utente e assistente, casella di testo che si allarga automaticamente scrivendo.
- **Funzionamento offline:** Bootstrap, le icone e il font sono copiati in locale nel progetto invece che caricati da un servizio esterno (CDN): una volta installate le dipendenze, l'app funziona anche senza connessione a Internet.

2. Architettura dell'applicazione

ConversAI segue un'architettura a tre livelli:

- **Client:** un'unica pagina HTML (`index.html`), single-page application. Tutta la logica di interfaccia è salvata in `public/app.js`.
- **Server:** un'unica applicazione Express (`server.js`) che espone esclusivamente un'API REST in JSON (vedi sezione 6) più i file statici del client.
- **Persistenza:** un singolo file SQLite (`data/conversai.sqlite`), letto e scritto tramite il modulo `database.js`.

Il client comunica con il server esclusivamente tramite chiamate `fetch` asincrone verso `/api/...`

3. Struttura del database

Il database SQLite (`data/conversai.sqlite`) contiene tre tabelle collegate gerarchicamente: un progetto contiene conversazioni, una conversazione contiene messaggi.

- **projects**(id: INTEGER, nome: TEXT, descrizione: TEXT, data_creazione: TEXT, data_aggiornamento: TEXT)
- **conversations**(id: INTEGER, *project_id: INTEGER, topic: TEXT, data_creazione: TEXT, data_aggiornamento: TEXT)
- **messages**(id: INTEGER, *conversation_id: INTEGER, ruolo: TEXT, contenuto: TEXT, data_creazione: TEXT)

SQLite non ha un tipo DATE/DATETIME nativo. I campi denominati con `data_` sono in realtà TEXT contenenti una data in formato ISO 8601 (prodotta con `Date.toISOString()` lato server). Le chiavi esterne sono attive (PRAGMA foreign_keys = ON) e configurate con ON DELETE CASCADE: eliminando un progetto vengono eliminate automaticamente tutte le sue conversazioni e, a cascata, tutti i messaggi in esse contenuti, senza query di cancellazione separate scritte a mano.

4. Istruzioni operative

Con Node.js installato, dalla cartella del progetto: `npm install` (installa le dipendenze e copia Bootstrap, icone e font in locale), opzionalmente `npm run init-db` per un set di dati di esempio, poi `npm start` (o `npm run dev` in sviluppo, con riavvio automatico) per avviare il server su `http://localhost:3000`.

5. Struttura del progetto

```
conversai/
|-- data/
|   |-- conversai.sqlite      # database SQLite (creato/aggiornato automaticamente)
|-- node_modules/            # dipendenze installate da npm
|-- public/                   # tutto ciò che il browser scarica direttamente
|   |-- index.html            # struttura della pagina (unica pagina dell'app)
|   |-- app.js                # logica dell'interfaccia (Fetch API verso il backend)
|   |-- style.css              # stile personalizzato sopra Bootstrap
|   |-- vendor/               # Bootstrap, Bootstrap Icons e font Inter, copiati in locale
|       |-- bootstrap/
|       |-- bootstrap-icons/
|       |-- inter/
|-- scripts/
|   |-- copy-vendor.js        # copia Bootstrap/Icons/Inter da node_modules a public/vendor
|-- utils/
|   |-- simulator.js          # genera le risposte "finte" dell'assistente
|-- database.js               # apertura del database e funzioni di accesso ai dati
|-- init-db.js                # script facoltativo per creare dati di esempio
|-- server.js                 # app Express: definisce tutte le rotte API
|-- package.json              # dipendenze e comandi npm (start, dev, init-db, postinstall)
-- package-lock.json
```

6. Rotte API (`server.js`)

Tutte le rotte hanno il prefisso comune `/api` (omesso qui sotto per brevità, tranne nella riga dei percorsi non riconosciuti); rispondono tutte in JSON e usano la chiave `errore` per segnalare gli errori.

Metodo	Percorso	Descrizione	Corpo della richiesta	Risposte
GET	<code>/projects</code>	Elenco progetti, dal più recente	—	200, 500
POST	<code>/projects</code>	Crea un progetto	{ nome, descrizione? }	201, 400, 500
GET	<code>/projects/:id</code>	Dettaglio di un progetto	—	200, 404, 500
PUT	<code>/projects/:id</code>	Modifica nome/descrizione	{ nome, descrizione? }	200, 400, 404, 500
DELETE	<code>/projects/:id</code>	Elimina un progetto (a cascata conversazioni e messaggi)	—	204, 404, 500
GET	<code>/projects/:id</code> <code>/conversations</code>	Conversazioni di un progetto	—	200, 404, 500
POST	<code>/projects/:id</code> <code>/conversations</code>	Crea una conversazione nel progetto	{ topic }	201, 400, 404, 500
GET	<code>/conversations/:id</code>	Dettaglio di una conversazione	—	200, 404, 500
PUT	<code>/conversations/:id</code>	Modifica il topic	{ topic }	200, 400, 404, 500

Metodo	Percorso	Descrizione	Corpo della richiesta	Risposte
DELETE	<code>/conversations/:id</code>	Elimina una conversazione (a cascata i messaggi)	—	204, 404, 500
GET	<code>/conversations/:id/messages</code>	Messaggi di una conversazione, in ordine cronologico	—	200, 404, 500
POST	<code>/conversations/:id/messages</code>	Invia un messaggio utente e riceve la risposta simulata	<code>{ contenuto }</code>	201, 400, 404, 500
—	<code>/api/*</code> non riconosciuto	Endpoint inesistente	—	404
GET	<code>/</code> e altri file statici	Serve i contenuti di <code>public/</code> (pagina, script, CSS, vendor)	—	200, 404

La risposta a `POST /api/conversations/:id/messages` restituisce entrambi i messaggi appena creati in un'unica chiamata: `{ messaggioUtente, messaggioAssistente }`, così il frontend può mostrarli subito senza dover rifare una richiesta.

7. Flusso di comunicazione front-end/back-end

Ogni interazione dell'utente segue lo stesso schema generale, illustrato qui con l'invio di un messaggio (il caso più completo):

1. L'utente scrive un messaggio e preme Invio (o clicca il pulsante di invio); `app.js` intercetta l'evento `submit` del form.
2. `app.js` chiama `chiamaApi(...)`, che esegue una `fetch` POST verso `/api/conversations/:id/messages` con il testo nel corpo della richiesta, in JSON.
3. `server.js` riceve la richiesta, verifica che la conversazione esista (altrimenti risponde 404) e che il campo `contenuto` sia valido (altrimenti 400).
4. `server.js` chiama `database.js` per salvare il messaggio dell'utente, poi chiama `utils/simulator.js` per generare la risposta automatica, poi richiama `database.js` per salvarla e per aggiornare la data dell'ultimo aggiornamento della conversazione.
5. Solo quando tutte queste operazioni asincrone sono andate a buon fine, `server.js` risponde al client con `{ messaggioUtente, messaggioAssistente }` e il codice di stato 201.
6. `app.js` riceve la risposta e aggiunge subito i due messaggi nell'area della conversazione, senza una seconda richiesta per rileggere l'intero storico, riducendo così le chiamate di rete necessarie.

Tutte le altre operazioni (creare/rinominare/eliminare progetti e conversazioni) seguono lo stesso schema: richiesta HTTP da `app.js` tramite `chiamaApi`, validazione ed eventuale accesso al database lato server, risposta JSON, aggiornamento della sola parte di interfaccia interessata lato client. Il client non mantiene mai una copia dei dati più a lungo di quanto serva per disegnare la schermata corrente: a ogni azione rilevante richiede di nuovo i dati aggiornati al server, tranne nel caso ottimizzato del punto 6.

8. Spiegazione dei file principali

`server.js`

Entry point Express: attiva `express.json()` e definisce le tredici rotte (sezione 6). Ogni rotta verifica prima l'esistenza della risorsa “genitore”, valida i dati in ingresso, poi opera sul database e risponde con lo stato HTTP appropriato. Serve anche i file statici di `public/` e gestisce con 404 le richieste `/api/...` non riconosciute.

`database.js`

Apri la connessione SQLite, crea le tre tabelle se mancanti e attiva `PRAGMA foreign_keys = ON` con `ON DELETE CASCADE`. Espone una funzione a callback per ogni operazione CRUD su progetti, conversazioni e messaggi, usata da

`server.js`.

`init-db.js`

Script opzionale (`npm run init-db`) che inserisce un progetto, una conversazione e un messaggio di esempio solo se il database è ancora vuoto.

`utils/simulator.js`

Funzione `generateAssistantReply`: applica cinque regole in ordine (saluto, ringraziamento, richiesta d'aiuto, domanda, eco di default) per generare la risposta simulata dell'assistente.

`scripts/copy-vendor.js`

Hook `postinstall`: copia Bootstrap, Bootstrap Icons e i soli pesi necessari del font Inter da `node_modules/` a `public/vendor/`, per l'uso offline.

`public/index.html`

Unica pagina della SPA: struttura header, sidebar, area messaggi, le modali di creazione/rinomina e il toast di errore; carica Bootstrap, Inter, lo stile personalizzato e infine `app.js`.

`public/app.js`

Logica dell'interfaccia in JavaScript vanilla: `chiamaApi` centralizza le chiamate di rete, un solo listener delegato gestisce i clic sulla sidebar, e il contenuto scritto dagli utenti è sempre inserito con `textContent` per evitare XSS.

`public/style.css`

Copre solo ciò che Bootstrap non offre già: tema scuro tramite `data-bs-theme`, stile dei messaggi, sidebar e auto-espansione della textarea.

`package.json`

Dipendenze (`express`, `sqlite3`, `bootstrap`, `bootstrap-icons`, `@fontsource/inter`) e script: `start`, `dev` (con `--watch`), `init-db`, `postinstall`.

9. Scelte progettuali

Bootstrap al posto del CSS scritto interamente a mano. A lezione il layout responsive viene insegnato scrivendo da zero CSS Grid e media query. Il progetto usa invece Bootstrap 5 come base, con `style.css` limitato a ciò che Bootstrap non copre già. Motivo: con un framework maturo si ottengono in tempi molto più brevi una griglia responsive coerente, componenti già testati e accessibili, e un aspetto visivo curato, tutti fattori che migliorano UI e UX rispetto a ricostruire ogni cosa a mano.

Componenti interattivi di Bootstrap (accordion, offcanvas, modale, toast). A lezione l'interattività si costruisce con `addEventListener` puro (mostra/nascondi manuale di elementi). Il progetto usa invece i componenti JavaScript inclusi in Bootstrap per aprire/chudere la lista dei progetti (accordion), il menu su mobile (offcanvas), le finestre di creazione/modifica (modale) e le notifiche di errore (toast). Motivo: questi componenti gestiscono da soli lo stato aperto/chiuso, gli attributi di accessibilità (`aria-expanded` e simili), il focus della tastiera e la chiusura con il tasto Esc.

Tema scuro tramite l'attributo `data-bs-theme`. Bootstrap 5.3 include il supporto nativo alla modalità scura: basta impostare l'attributo su `<html>` e ridefinire poche variabili (il colore primario, quello del testo dell'assistente) invece di riscrivere ogni componente (bottoni, form, modali) in versione scura.

Tre tabelle collegate da chiavi esterne, con cancellazione a cascata. Questo progetto richiede tre entità collegate gerarchicamente (un progetto contiene conversazioni, una conversazione contiene messaggi), quindi `database.js` usa chiavi esterne con `ON DELETE CASCADE` (attivate con `PRAGMA foreign_keys = ON`): eliminando un progetto, SQLite cancella da solo le conversazioni e i messaggi collegati. L'alternativa senza questa impostazione

avrebbe richiesto scrivere a mano, in JS, tre cancellazioni annidate ogni volta che si elimina un progetto o una conversazione.

Librerie front-end copiate in locale invece che caricate da CDN. Il modo più comune e rapido per usare Bootstrap (anche in molti tutorial) è un collegamento diretto a un CDN nell'HTML. Qui invece uno script dedicato (`scripts/copy-vendor.js`) copia i file già compilati di Bootstrap, Bootstrap Icons e del font Inter dentro `public/vendor/` dopo ogni `npm install`. Motivo: il progetto richiede di funzionare senza servizi esterni, e un CDN introdurrebbe una dipendenza di rete a runtime che verrebbe meno se l'app viene usata offline o il CDN è irraggiungibile.

node --watch al posto di nodemon per il riavvio automatico in sviluppo. Da Node.js 18 in poi questa funzionalità è integrata nativamente (`node --watch`), quindi il progetto la usa direttamente nello script `dev` senza aggiungere una dipendenza in più.

10. Limitazioni note

- **Nessuna autenticazione o gestione multiutente:** chiunque avvii l'applicazione localmente vede e può modificare tutti i progetti, le conversazioni e i messaggi presenti nel database. Non esiste un concetto di “utente proprietario” di un progetto.
- **Risposta dell'assistente non generata da un vero modello linguistico:** come descritto nella sezione 8 (`utils/simulator.js`), le risposte seguono cinque regole fisse basate su parole chiave; non c'è comprensione reale del contenuto scritto dall'utente.
- **Nessuna paginazione:** le liste di progetti, conversazioni e messaggi vengono sempre caricate per intero. Con un numero molto elevato di elementi le prestazioni dell'interfaccia potrebbero peggiorare.
- **Nessun test automatico:** il progetto non include una suite di test (unitari o di integrazione); la correttezza è stata verificata manualmente durante lo sviluppo.